

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	DSA0305	Course Title:	Natural Language Processing
CO Assessed:	CO4 – Precision	Assessment:	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	40% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	N.Adi Narayana	Reg. No.:	192525346
Section / Batch:	AIML	Date:	16-08-2026

Code of Conduct

I N.Adi Narayana (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Adi _____

Instructions

- Read each industry scenario carefully before answering.
- Analyze the given problem from a semantic analysis perspective.
- Provide structured and logical answers with proper justification.
- Support your analysis using examples from the given scenario wherever applicable.
- Use suitable diagrams, semantic representations, logical predicates, workflows, architectures, or tables whenever necessary.
- Clearly state assumptions if additional information is required.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze semantic interpretation of user queries, identify ambiguity in meaning representation, evaluate semantic processing limitations, and improve understanding in banking chatbot applications.	Q1
Syntax-Driven Semantic Analysis	Analyze multiple interpretations of spoken commands, evaluate parsing-related ambiguity, and assess semantic extraction in real-time voice assistant systems.	Q2
First-Order Predicate Calculus, Representing Linguistically Relevant Concepts, Syntax-Driven Semantic Analysis	Design a semantic processing architecture for healthcare reports, model relationships among entities and actions, represent linguistic concepts, and improve semantic extraction accuracy.	Q3

Annexure A – Industry Problem Solving Task

1. A company is developing an AI-powered customer support chatbot for banking services. The chatbot uses Context-Free Grammar (CFG) for parsing user queries. However, it faces issues such as:

Incorrect interpretation of ambiguous sentences

Failure to handle subject–verb agreement

Inefficient parsing for long queries

Example user query:

“Show me the transactions with the card from last month”

a) Analyze the ambiguity present in the given sentence using CFG

b) Evaluate the limitations of CFG in handling real-world conversational queries

c) Evaluate and propose an improved solution using PCFG, feature structures, or Earley parsing

d) Justify how your proposed method improves accuracy and efficiency in an industry setting

2. A voice assistant system (like Alexa/Siri-type applications) processes spoken commands using parsing techniques. The system currently uses top-down parsing, but faces:

Backtracking issues

Poor handling of incomplete or ambiguous input

Difficulty in real-time response

Example command:

“Book a flight to Delhi with a window seat”

a) Analyze how the given sentence can lead to multiple parse structures

b) Analyze the limitations of top-down parsing in this real-time application

c) Analyze how Earley parsing can improve handling of ambiguity and partial inputs

d) Compare and analyze the performance of top-down vs Earley parsing for such industry use cases

3. A healthcare company is developing an AI system to analyze patient medical reports and automatically extract critical information such as diagnosis, prescribed treatments, and follow-up actions.

The system must handle:

Complex and lengthy medical sentences

Ambiguity in sentence structure

Accurate subject-verb agreement

Relationships between medical actions and entities (sub-categorization)

Real-time processing for hospital use

Example input:

"The doctor who reviewed the patient last week recommends starting medication and scheduling a follow-up visit in Chennai."

The current system, based only on basic Context-Free Grammar (CFG), fails to correctly interpret relationships and produces inconsistent outputs.

a) Design a complete NLP architecture integrating:

CFG for syntactic structure

Probabilistic CFG (PCFG) for ambiguity resolution

Feature Structures for agreement handling

Efficient parsing techniques

b) Develop a step-by-step workflow showing how the system processes the given medical sentence from input to structured output (diagnosis, action, location).

c) Create a method to:

Incorporate sub-categorization frames for medical verbs

Handle real-time data processing

Improve accuracy and scalability in a hospital environment

Q1. Banking Customer Support Chatbot Using CFG

Given Scenario

A banking company is developing an AI-powered customer support chatbot that uses Context-Free Grammar (CFG) to parse customer queries. The system faces problems such as incorrect interpretation of ambiguous sentences, failure to handle subject–verb agreement, and inefficient parsing of long queries.

The example query is:

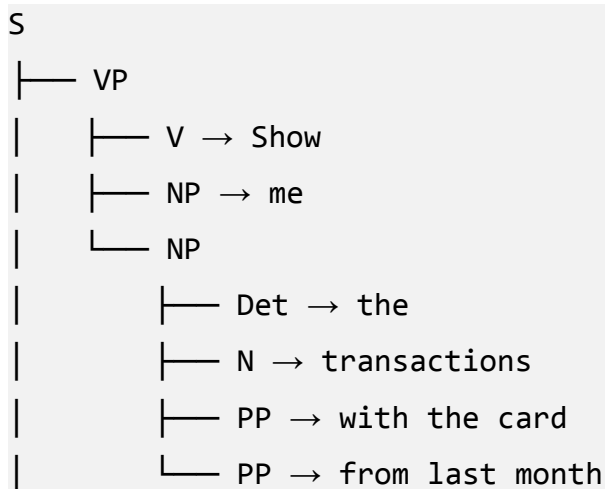
“Show me the transactions with the card from last month.”

The assessment specifically asks for ambiguity analysis, CFG limitations, an improved solution using PCFG/feature structures/Earley parsing, and industry justification.

a) Ambiguity in the Given Sentence Using CFG

The sentence “Show me the transactions with the card from last month.” contains syntactic and semantic ambiguity.

The main ambiguity is associated with the phrase “with the card” and the phrase “from last month.”



The semantic interpretation is:

```
SHOW(
    transactions(
        card = specified_card,
        time = last_month
```

```
)  
)
```

Interpretation 2 – “From last month” modifies the card

Another possible interpretation is that “from last month” is associated with the card rather than the transactions. For example:

```
transactions [with the card [from last month]]
```

Ambiguous Attachment

The ambiguity mainly comes from prepositional phrase attachment.

```
      transactions  
      /           \  
with the card   from last month
```

The parser must determine whether the PP modifies the noun phrase “transactions” or another constituent. A simple CFG can generate multiple valid parse structures without understanding which interpretation is more appropriate in the banking context.

Semantic Representation

A better semantic representation is:

```
Transaction(x)  
UsedWith(x, Card)  
OccurredDuring(x, LastMonth)  
RequestedBy(User)
```

Therefore:

```
Find x such that:  
Transaction(x)  
AND UsedWith(x, Card)  
AND OccurredDuring(x, LastMonth)
```

This representation removes much of the ambiguity by explicitly defining the relationships.

b) Limitations of CFG for Real-World Conversational Queries

CFG is useful for representing grammatical structure, but it has several limitations in a banking chatbot.

1. CFG does not represent probabilities. For example: “The customer checked the account with the card.” The phrase “with the card” may have multiple possible attachments. CFG treats valid grammatical structures similarly unless additional mechanisms are introduced.

```
S → NP VP  
NP → Det N  
VP → V NP
```

but these rules do not inherently ensure that the verb agrees with the subject. A feature-based representation can solve this:

```
NP[NUM=singular] → customer  
VP[NUM=singular] → checks NP
```

and:

```
NP[NUM=plural] → customers  
VP[NUM=plural] → check NP
```

3. Long sentences increase parsing complexity

Real banking queries may contain: transaction descriptions, account numbers, dates, card information, conditions, multiple requests, references to previous conversations.

For example: “Show me all transactions made using my debit card in Chennai during the first week of last month and tell me which ones were above ₹10,000.” A basic CFG can produce a very large number of possible structures.

4. CFG does not understand context

A banking chatbot must understand words according to context. For example: “Show my balance.” Here, “balance” refers to a banking account balance. In another domain, balance could refer to physical balance or mathematical balance. CFG mainly describes syntax and does not provide sufficient semantic or contextual information.

5. Handling conversational language

Users may provide incomplete queries: “Transactions last month.” or “My card transactions?” These are common in conversational systems but do not necessarily follow complete grammatical structures. A rigid CFG may fail to process such inputs effectively.

c) Improved Solution Using PCFG, Feature Structures and Earley Parsing

A better banking NLP system should combine several techniques instead of relying only on CFG.

Proposed Architecture



1. PCFG

A Probabilistic Context-Free Grammar (PCFG) assigns probabilities to grammar rules. For example:

$S \rightarrow NP\ VP$	0.95
$NP \rightarrow Det\ N$	0.70
$NP \rightarrow NP\ PP$	0.30

VP → V NP 0.80

NP → NP PP 0.25

If multiple parse trees are possible, the system calculates the probability of each parse and selects the most probable interpretation. For the given query, the banking-specific interpretation:

transactions

└─ with card

└─ from last month

would receive a higher probability than unlikely alternatives if the model has been trained on banking conversations.

2. Feature Structures

Feature structures can represent grammatical information such as:

Feature	Example
Number	Singular / Plural
Person	First / Second / Third
Tense	Present / Past
Case	Subject / Object
Semantic Type	Account / Card / Transaction
Time	Last month
Intent	Transaction search

For example:

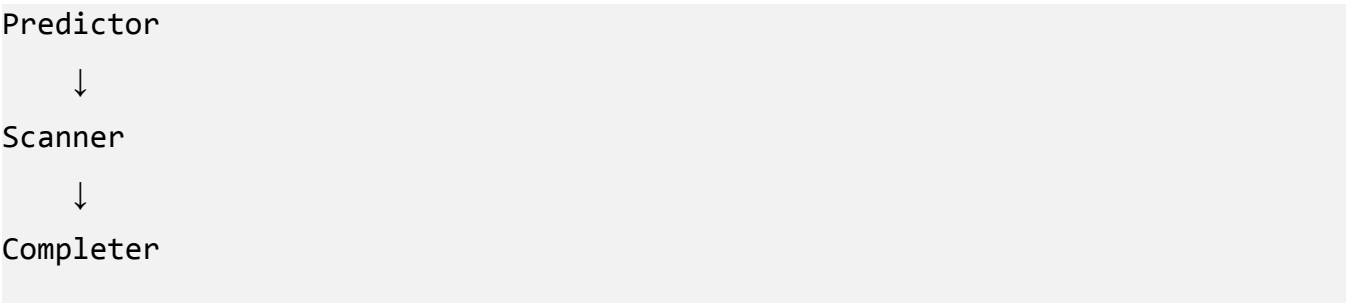
NP:

```
{  
  TYPE: transaction,  
  NUMBER: plural,  
  TIME: last_month,  
  CARD: specified_card  
}
```

The parser can use these features to reject incompatible interpretations.

3. Earley Parsing

Earley parsing is suitable for this application because it can handle: ambiguous grammars, incomplete sentences, left-recursive rules, complex sentence structures. It uses three major operations:



For the query “Show me the transactions with the card from last month,” Earley parsing can maintain multiple possible interpretations without immediately committing to one parse. The PCFG can then rank those interpretations.

d) Justification for Industry Use

The proposed solution improves the banking chatbot in several ways.

Problem	Basic CFG	Proposed Method
Ambiguous sentences	Poor handling	PCFG ranks alternatives
Agreement	Limited	Feature structures
Long queries	Can become inefficient	Earley parsing
Incomplete input	Weak	Earley handles partial structures
Context	Limited	Semantic layer
Banking entities	Not specialized	Domain-specific semantic extraction
Accuracy	Moderate	Combined parsing approach
Scalability	Limited	Modular architecture

Example

Input: “Show me the transactions with the card from last month.”

Output:

Intent: Transaction_Search

Entity:

Card = specified card

Time:

Last Month

Operation:

Retrieve Transactions

The chatbot can then query the banking database and return only the relevant transactions.

Conclusion

CFG provides the basic syntactic foundation, but it is insufficient by itself for real-world banking conversations. Combining Earley parsing, PCFG and feature structures provides better ambiguity handling, grammatical validation and efficient parsing. This approach is more suitable for a banking chatbot where incorrect interpretation can lead to incorrect financial information being displayed.

Q2. Voice Assistant – Parsing Spoken Commands

Given Scenario

A voice assistant processes spoken commands using parsing techniques. The current system uses top-down parsing but suffers from backtracking, poor handling of incomplete or ambiguous input, and difficulty providing real-time responses.

The given command is:

“Book a flight to new Delhi with a window seat.”

The assessment requires analysis of multiple parse structures, top-down limitations, Earley parsing, and a comparison of both approaches.

a) Multiple Parse Structures

The sentence can be represented as: “Book a flight to Delhi with a window seat.” A possible basic structure is:

```
S
├── VP
│   ├── V → Book
│   └── NP
│       ├── Det → a
│       ├── N → flight
│       ├── PP → to Delhi
│       └── PP → with a window seat
```

Here: Book → action, a flight → object, to Delhi → destination, with a window seat → seat preference.

The semantic representation is:

```
BOOK(
    Flight,
    Destination = Delhi,
    SeatPreference = Window
)
```

Ambiguity

The phrase “with a window seat” may have attachment ambiguity. It could be interpreted as a property of the flight:

```
flight [with a window seat]
```

or as an additional booking constraint:

```
Book flight
```

```
  + Destination = Delhi
```

```
  + Seat = Window
```

In a voice assistant, the second interpretation is normally intended.

b) Limitations of Top-Down Parsing

Top-down parsing starts from the start symbol and tries to derive the input. For example:

```
S
```

```
↓
```

```
VP
```

```
↓
```

```
V NP
```

```
↓
```

```
Book NP
```

It continues expanding grammar rules until it matches the input.

1. Backtracking

If an incorrect grammar rule is selected, the parser may need to return to an earlier decision. For an ambiguous sentence, this causes repeated processing.

```
Parse A
```

```
↓
```

```
Failure
```

```
↓
```

```
Backtrack
```

```
↓
```

```
Parse B
```

```
↓
```

```
Failure
```



Parse C

This is undesirable in voice assistants because users expect immediate responses.

2. Difficulty with Ambiguity

A spoken command can have several possible interpretations. The system must determine:

Action = Book

Destination = Delhi

Seat = Window

Top-down parsing can spend unnecessary time exploring alternatives.

3. Incomplete Speech

Voice assistants frequently receive incomplete input. Examples: “Book a flight to...” or “Book Delhi...”

A top-down parser may struggle when the expected grammatical structure is incomplete.

4. Real-Time Constraints

Voice assistants need low latency. The system typically performs:

Speech



Speech Recognition



Text



Parsing



Intent Detection



Action

If parsing requires excessive backtracking, response time increases.

c) Earley Parsing for Ambiguous and Partial Input

Earley parsing is a better choice for this type of application because it can efficiently process general CFGs and maintain multiple possible interpretations. It consists of three primary operations.

1. Predictor

The parser predicts possible grammar rules. For example:

$S \rightarrow VP$

$VP \rightarrow V\ NP$

$NP \rightarrow Det\ N\ PP$

2. Scanner

The parser matches grammar predictions against the actual words.

Book

a

flight

to

Delhi

with

a

window

seat

3. Completer

When a constituent has been successfully recognized, Earley parsing uses it to complete larger structures. The process can be visualized as:

Input

↓

Predict possible structures

↓

Scan words

↓

Complete recognized structures

↓

Maintain alternative parses

↓

Select best semantic interpretation

Earley parsing does not need to immediately discard alternative interpretations. This is particularly useful when the input is ambiguous or incomplete.

d) Top-Down vs Earley Parsing

Feature	Top-Down Parsing	Earley Parsing
Parsing direction	Starts from grammar start symbol	Processes input while maintaining states
Backtracking	Can be significant	Avoids repeated exploration through chart states
Ambiguity	More difficult	Handles ambiguity naturally
Incomplete input	Weak	Better suited
Left recursion	Problematic in many top-down implementations	Supported
Real-time use	Can suffer from latency	More suitable for complex input
Memory	Can repeatedly recompute	Uses chart/state storage
Grammar flexibility	More restricted	General CFG

Example Processing

For “Book a flight to Delhi with a window seat,” the system can produce:

Intent:

BOOK_FLIGHT

Destination:

Delhi

Seat Preference:

Window

Instead of treating “with a window seat” merely as a syntactic phrase, the semantic layer converts it into a booking constraint.

Industry Recommendation

A practical voice assistant should use:

Speech Recognition



Text Normalization



Earley Parser



Possible Parse Structures



Semantic Ranking



Intent + Entity Extraction



Dialogue Manager



Flight Booking API



Voice Response

For example, User: “Book a flight to Delhi with a window seat.”

System interpretation:

Intent = BookFlight

Destination = Delhi

SeatPreference = Window

The dialogue manager can then ask for missing information such as: “What date would you like to travel?” This makes the system conversational instead of requiring the user to provide every detail in one sentence.

Conclusion

Top-down parsing is simple and useful for restricted grammars, but it can become inefficient when dealing with ambiguity, incomplete input and real-time speech. Earley parsing is more appropriate for voice assistants because it can handle general CFGs, partial input and multiple interpretations more effectively. Combining Earley parsing with semantic interpretation provides a more robust voice-based system.

Q3. Healthcare Medical Report Semantic Processing System

Given Scenario

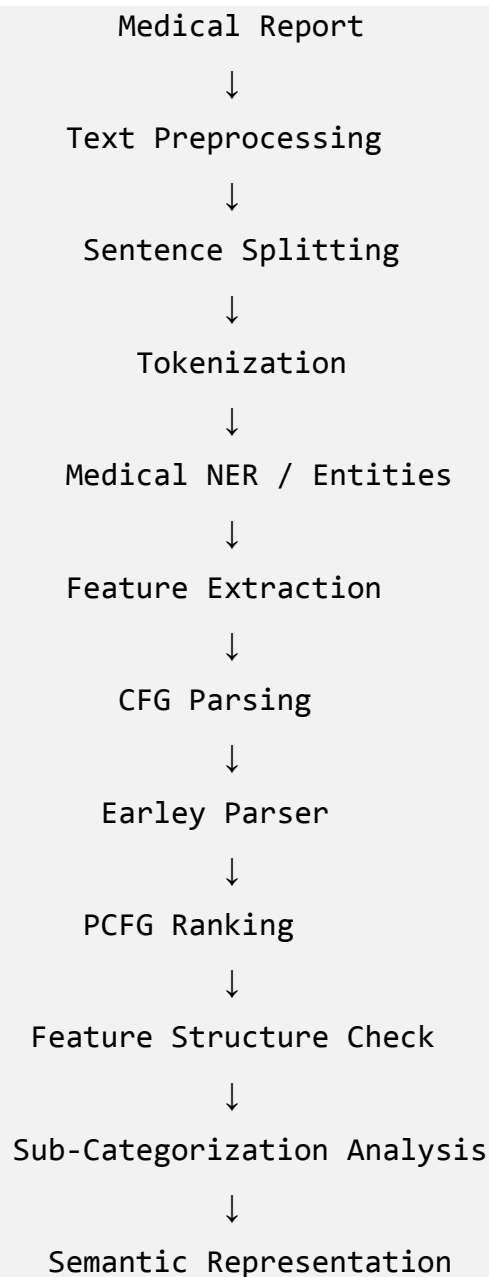
A healthcare company wants to build an NLP system that extracts important information from medical reports, including diagnosis, prescribed treatments and follow-up actions.

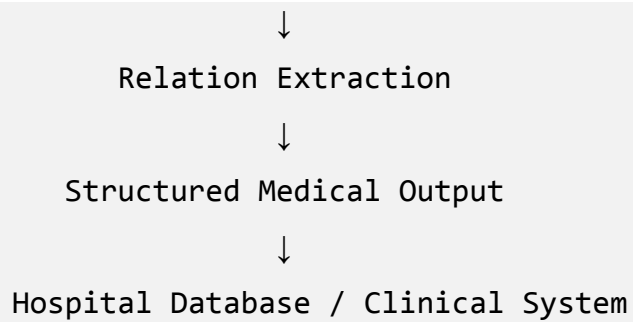
The example sentence is:

“The doctor who reviewed the patient last week recommends starting medication and scheduling a follow-up visit in Chennai.”

a) Complete NLP Architecture

A suitable architecture should combine CFG, PCFG, feature structures and efficient parsing.





1. Text Preprocessing

The first stage cleans and normalizes the medical report. Operations include: sentence segmentation, tokenization, abbreviation handling, punctuation processing, medical terminology normalization. For example:

Doctor → Medical Professional
Medication → Treatment
Follow-up visit → Clinical Follow-up
Chennai → Location

2. Medical Entity Recognition

The system identifies important entities. For the given sentence:

Text	Entity Type
doctor	Medical Professional
patient	Patient
last week	Time
medication	Treatment
follow-up visit	Medical Action
Chennai	Location

3. CFG for Syntactic Structure

CFG can provide the basic syntactic structure. A simplified grammar is:

$S \rightarrow NP \ VP$

$NP \rightarrow Det \ N$

$NP \rightarrow NP \ RelativeClause$

$NP \rightarrow Det \ N \ PP$

VP → V NP

VP → V GerundPhrase

VP → V NP Conjunction VP

RelativeClause → RelPronoun VP

PP → P NP

For the sentence:

S

```
|— NP
|   |— The doctor
|   |— Relative Clause
|       |— who reviewed the patient last week
|
|— VP
    |— recommends
    |— actions
        |— starting medication
        |— scheduling a follow-up visit in Chennai
```

This provides the syntactic foundation.

4. PCFG for Ambiguity Resolution

Medical reports may contain complex sentences where several syntactic interpretations are possible. A PCFG assigns probabilities to competing parse structures. For example:

Parse 1 = 0.80

Parse 2 = 0.15

Parse 3 = 0.05

The system selects the most probable interpretation while retaining alternatives when confidence is low. This is useful for clinical NLP because incorrect interpretation of a medical statement can result in incorrect information extraction.

5. Feature Structures for Agreement

Feature structures allow the parser to represent grammatical properties. Example:

Doctor:

```
{  
  category: noun,  
  number: singular,  
  semantic_type: medical_professional  
}
```

Verb:

```
recommends:  
{  
  tense: present,  
  number: singular,  
  semantic_type: medical_action  
}
```

The system checks:

```
Doctor [singular]  
    ↓  
recommends [singular]
```

Therefore, the subject–verb agreement is valid.

6. Efficient Parsing

For complex medical sentences, Earley parsing can be integrated. It helps process: long sentences, ambiguous structures, recursive structures, partial input. Instead of repeatedly rebuilding parse trees, the parser maintains intermediate states in a chart. This makes the system more appropriate for real-time clinical environments.

b) Step-by-Step Workflow

The given input is: “The doctor who reviewed the patient last week recommends starting medication and scheduling a follow-up visit in Chennai.”

Step 1 – Input

The doctor who reviewed the patient last week recommends
starting medication and scheduling a follow-up visit in Chennai.

Step 2 – Tokenization

The sentence is divided into tokens:

```
The | doctor | who | reviewed | the | patient |  
last | week | recommends | starting | medication |  
and | scheduling | a | follow-up | visit | in | Chennai
```

Step 3 – Entity Detection

The system identifies:

```
Doctor      → Medical Professional  
Patient     → Patient  
Last week   → Time  
Medication  → Treatment  
Follow-up   → Medical Action  
Chennai     → Location
```

Step 4 – Syntactic Parsing

The parser identifies:

Subject:

The doctor who reviewed the patient last week

Main Verb:

recommends

Actions:

1. starting medication
2. scheduling a follow-up visit in Chennai

The relative clause “who reviewed the patient last week” modifies “doctor.” Therefore:

Doctor

```
└─ reviewed  
    └─ patient  
        └─ last week
```

Step 5 – Feature Checking

The system checks:

Subject = doctor

Number = singular

Verb = recommends

Number = singular

The features agree.

Step 6 – Semantic Analysis

The system identifies the relationships:

Doctor → recommends → starting medication

Doctor → recommends → scheduling follow-up visit

Follow-up visit → Location → Chennai

Step 7 – Predicate Representation

The sentence can be represented using logical predicates.

Doctor(d)

Patient(p)

Medication(m)

Visit(v)

Chennai(c)

Reviewed(d, p, LastWeek)

Recommends(d, StartMedication(m))

Recommends(d, Schedule(v))

Location(v, c)

A simplified combined representation is:

$\exists d, p, m, v, c$ [

Doctor(d)

$\wedge$ Patient(p)

$\wedge$ Reviewed(d, p, LastWeek)

```
    ^ Recommends(d,StartMedication(m))
    ^ Recommends(d,Schedule(v))
    ^ FollowUpVisit(v)
    ^ Location(v,c)
    ^ Chennai(c)
]
```

This representation makes the relationships between entities and actions explicit.

Step 8 – Structured Output

The final output can be stored as:

This structured representation can then be used by a hospital information system.

c) Sub-Categorization Frames for Medical Verbs

Sub-categorization describes the arguments that a verb normally requires.

Verb: “recommend”

A useful frame is:

recommend

Subject: Medical Professional

Object/Complement: Treatment or Action

Example: “Doctor recommends medication.” Semantic representation:

Recommend(Doctor, Medication)

Verb: “review”

review

Subject: Medical Professional

Object: Patient / Medical Report

Example: “Doctor reviewed patient.” Representation:

Review(Doctor, Patient)

Verb: “schedule”

schedule

Subject: Medical Professional/System

Object: Appointment/Visit

Location: optional

Time: optional

Example: "Doctor schedules follow-up visit in Chennai." Representation:

Schedule(Doctor, FollowUpVisit)

Location(FollowUpVisit, Chennai)

Medical Verb Frame Table

Sub-categorization frames improve extraction because the system knows what entities are expected around a particular medical verb.

Real-Time Processing in a Hospital

A hospital NLP system must provide information quickly. A suitable architecture is:

Medical Report



Streaming Input



Preprocessing



Medical NER



Efficient Parser



Semantic Extraction



Confidence Score



Clinical Database

Techniques for Real-Time Processing

1. Incremental Parsing – Instead of waiting for an entire document, the system processes sentences as they arrive.

2. Parallel Processing – Different reports can be processed simultaneously using multiple processing workers.

Report 1 → Worker 1

Report 2 → Worker 2

Report 3 → Worker 3

Report 4 → Worker 4

3. Caching – Frequently used medical terms and grammar structures can be cached.

4. Domain-Specific Vocabulary – The system should maintain medical dictionaries containing: diseases, medicines, treatments, procedures, body parts, clinical abbreviations.

5. Confidence Scoring – Each extracted relationship can have a confidence score. Example:

Diagnosis: Pneumonia → 0.96

Medication: Antibiotic → 0.94

Location: Chennai → 0.99

Low-confidence outputs can be flagged for human review.

Improving Accuracy and Scalability

A hospital NLP system should not depend only on CFG. The proposed approach combines:

CFG

+

PCFG

+

Feature Structures

+

Earley Parsing

+

Medical NER

+

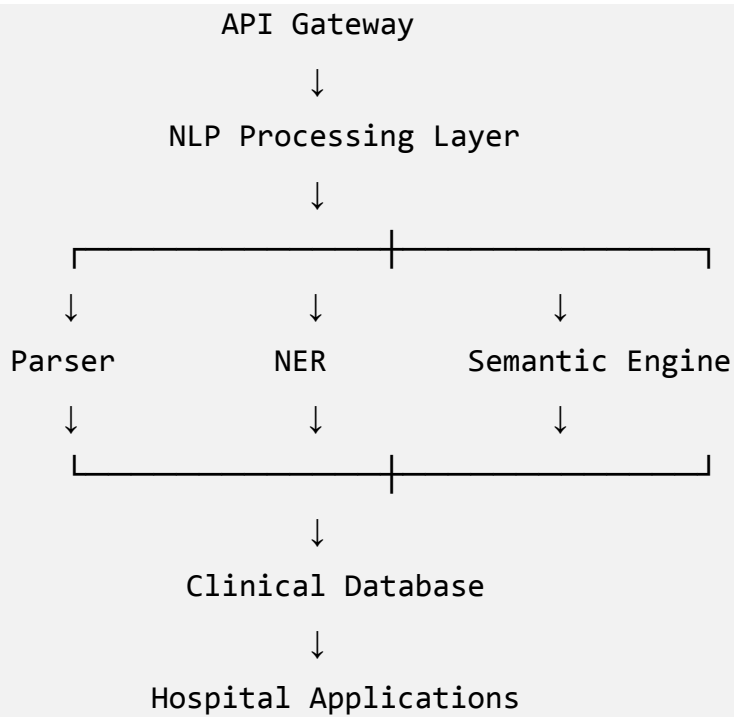
Sub-Categorization

+

Semantic Representation

Accuracy Improvements Scalability Improvements

The system can use a modular architecture:



This allows individual components to be scaled independently. For example, if the number of incoming medical reports increases, additional parser and NER workers can be added without redesigning the complete system.

Overall Comparison of the Three Solutions

Final Conclusion

The three industry scenarios demonstrate that CFG alone is not sufficient for modern NLP applications. CFG provides an important syntactic foundation, but real-world systems must also deal with ambiguity, incomplete input, grammatical agreement, domain-specific terminology and semantic relationships.

For the banking chatbot, combining CFG, PCFG, feature structures and Earley parsing improves interpretation of ambiguous transaction queries.

For the voice assistant, Earley parsing is more suitable than a purely top-down approach because voice commands may be ambiguous, incomplete and time-sensitive.

For healthcare NLP, a complete architecture should combine CFG, PCFG, feature structures, efficient parsing, medical entity recognition, sub-categorization frames and semantic representations. The extracted information can then be converted into structured clinical data such as diagnosis, treatment, action and location.

Thus, a practical NLP system should use syntax + probability + linguistic features + semantic analysis rather than depending on a single parsing technique. This provides better accuracy, robustness, efficiency and scalability for real-world industry applications.

Rubrics for Calculation

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding ; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Q. No. / Criteria Level	Understanding of Industry Problem	Application of Engineering Concepts	Solution Design	Use of Tools	Analysis	Professionalism	Sub. Total	Total %
1								
2								
3								

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

ANSWERS